

助教模擬考期末複習

Problem 1, 2

目錄

UVa 10066 - The Twin Towers	2
題目敘述	2
題解說明	2
完整程式碼	2
詳細知識點說明	3
輸入	3

UVa 10066 - The Twin Towers

<https://zerojudge.tw/ShowProblem?problemid=a133>

<https://vjudge.net/problem/UVA-10066>

題目敘述

給定兩座雙子星塔，分別由 N 個與 M 個不同半徑的圓形磁磚疊成。為了讓兩座塔看起來一樣，必須從兩座塔中抽掉部分磁磚，使得剩下的磁磚序列（半徑大小與順序）完全一致。

目標： 求出保留下來的磁磚數量最多可以是多少？（多筆測資，當 $N = 0$ 且 $M = 0$ 時結束程式）。

題解說明

這是一道標準的 **最長共同子序列 (LCS, Longest Common Subsequence)** 問題，只是將常見的字串比對換成了整數陣列的比對。

關於 LCS 的狀態轉移，我們直接使用下列公式來建構二維的 DP 表格：

```
for (int i = 1; i <= N; ++i)
    for (int j = 1; j <= M; ++j)
    {
        if (N1[i] == N2[j]) { DP[i][j] = DP[i - 1][j - 1] + 1; } // green
        else { DP[i][j] = max(DP[i - 1][j], DP[i][j - 1]); } // orange
    }
```

(註：關於 LCS 的底層數學原理、子問題拆解與二維表格的視覺化，請參閱後續的[詳細知識點說明](#)。)

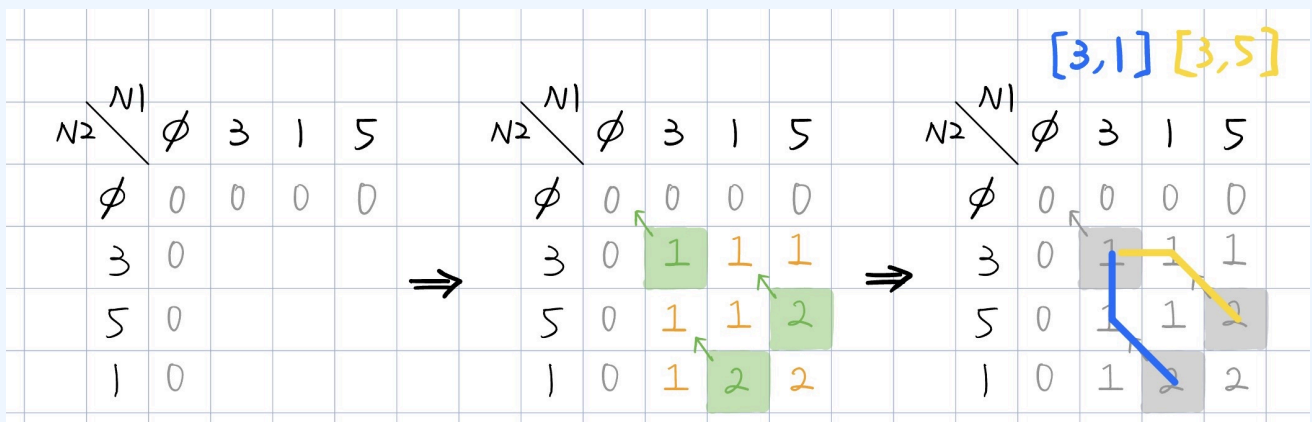
實作要點：

- 宣告大小為 $(N + 1) \times (M + 1)$ 的 DP 陣列，邊界預設為 0。
- 迴圈從 1 遍歷至 N 與 M ，因為 DP 表格有留一格邊界，所以存取 $N1$ 與 $N2$ 時索引要減一 ($i - 1, j - 1$)。
- 最終答案即為表格最右下角的值 $DP[N][M]$ 。
- 輸出格式請注意，每筆測資的結果之間需要多一個空白行。

範例追蹤：

假設第一座塔 $N_1 = [3, 1, 5]$ ，第二座塔 $N_2 = [3, 5, 1]$

- 當比對到相同的磁磚 (如 3 對 3)： $DP[1][1] = DP[0][0] + 1$
- 當比對到不同的磁磚 (如 1 對 5)： $DP[2][2] = \max(DP[1][2], DP[2][1])$
- 最終最長相同序列為 $[3, 1]$ 或 $[3, 5]$ ，最大磁磚數為 2。



完整程式碼

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
```

```

4 using namespace std;
5
6 int main()
7 {
8     int N, M;
9     int testcase = 1;
10
11 while (cin >> N >> M and (N != 0 and M != 0))
12 {
13     vector<int> N1(N + 1);
14     for (int i = 1; i <= N; ++i) { cin >> N1[i]; }
15     vector<int> N2(M + 1);
16     for (int i = 1; i <= M; ++i) { cin >> N2[i]; }
17     vector<vector<int>> DP(N + 1, vector<int>(M + 1, 0));
18
19     // LCS algorithm
20     for (int i = 1; i <= N; ++i)
21         for (int j = 1; j <= M; ++j)
22         {
23             if (N1[i] == N2[j]) { DP[i][j] = DP[i - 1][j - 1] + 1; }
24             else { DP[i][j] = max(DP[i - 1][j], DP[i][j - 1]); }
25         }
26
27     cout << "Twin Towers #" << testcase++ << "\n";
28     cout << "Number of Tiles : " << DP[N][M] << "\n\n";
29 }
30
31 return 0;
32 }

```

詳細知識點說明

輸入